

## Trabajo Práctico Final Integrador Patrón Proxy/Singleton/Observer en entorno AWS

### Nota importante

Asegúrese de haber completado la instalación y configuración de pre-requisitos según lo indicado en la nota de aplicación “IS2\_NAPP\_Arquitectura AWS Python” antes de intentar ejecutar cualquier desarrollo producto de las especificaciones aquí indicadas.

### Problema

Como parte del despliegue de un sistema aplicativo es necesario desarrollar componentes para soportar la infraestructura de la misma. Para ello es necesario implementar clases que permitan proveer una serie de servicios corporativos para la adquisición de datos centralizados de éste aplicativo y el resto de la arquitectura, entre ellos:

- Datos oficiales de dirección.
- Número de CUIT.
- Número de teléfono de contacto.
- Identificador único de secuencia.

Se trabajará con la premisa de implementar tres funciones principalmente:

- Recuperar datos corporativos.
- Modificar datos corporativos.
- Listar database completo.

Para implementar la funcionalidad se utilizarán dos tablas implementadas en AWS-DynamoDB.

- **CorporateData.** Contiene los datos centralizados.
- **CorporateLog.** Contiene registros de los accesos y modificaciones realizados.

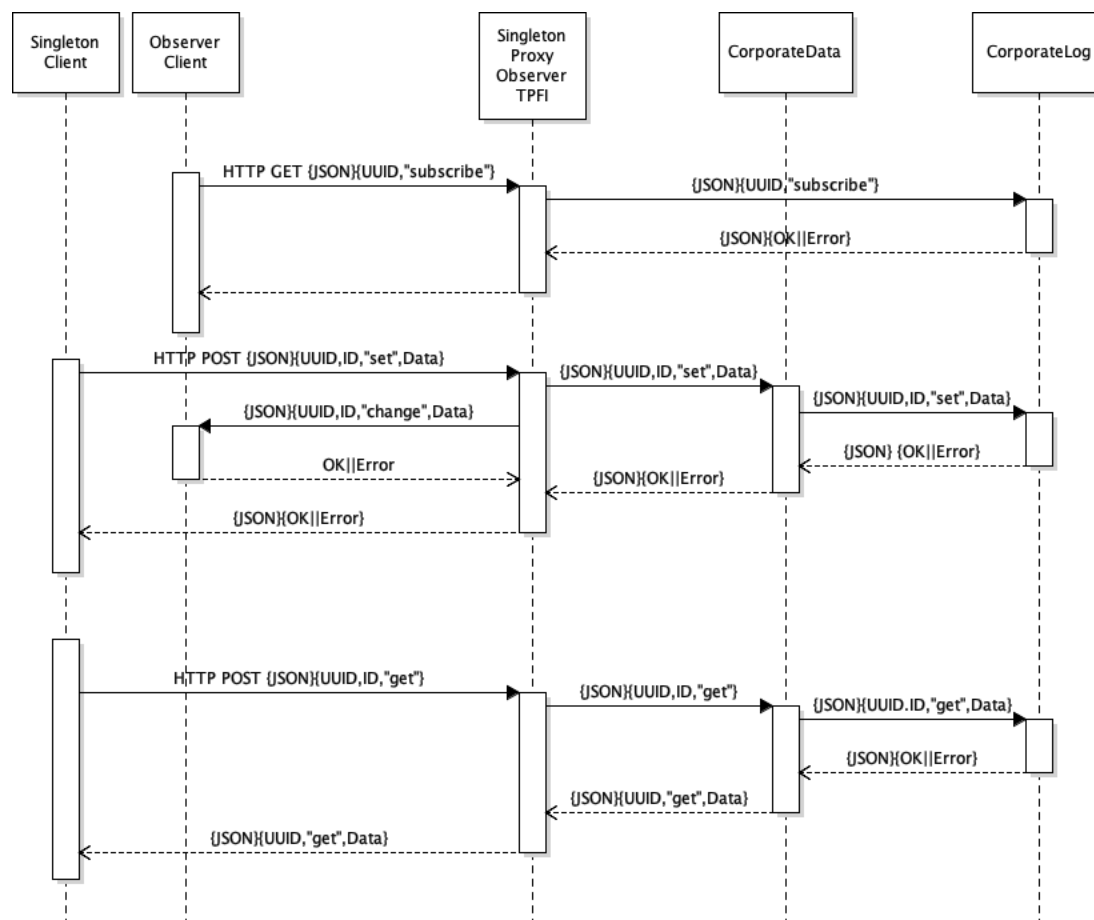


Ilustración 1 Diagrama de secuencia de aplicativos a implementar

El uso práctico de las funciones a implementar debe realizarse de acuerdo a la arquitectura mediante diferentes clases implementadas utilizando el lenguaje Python.

## Diseño de componentes

Se implementarán tres programas aplicativos en lenguaje Python.

- SingletonClient
- ObserverClient
- SingletonProxyObserverTPFI

La funcionalidad de los mismos se describe a continuación.

### SingletonClient

Se trata de un programa escrito en Python que puede hacer requerimientos de información sobre la base *CorporateData* o actualizar los datos de la misma.

Este programa operará como cliente utilizando el siguiente formato:

```
python singletonclient.py -i=input.json {-o=output.json}{-v}
```

El archivo JASON de entrada (*input.json*) tendrá un formato indicativo dado por:

```
{
  "UUID": "XXXXXXXXXXXXXXXXXXXXXXXXX",
  "ID": "ZZZZZZZZZZZZZZZZZZZZZZZZ",
  "ACTION" : "get"
}
```

El archivo de salida (*output.json*) es un argumento opcional, de no indicarse la salida se emitirá por salida estándar, de informarse se grabará en el archivo cuyo nombre se indica.

Para hacer lo primero genera un archivo JSON indicando:

- La identificación de la CPU desde la que ejecuta (UUID).
- La identificación del registro que quiere acceder (ID).
- Acción a realizar (“get”, “set” o “list”).
- Solo en el caso de acción “set” todos los campos del tuple CorporateData.

En respuesta recibirá un archivo JSON con:

- “get”: Registro de CorporateData solicitado o “Error”.
- “set”: Registro de CorporateData modificado o “Error”.
- “list”: Todos los registros de la tabla CorporateData o “Error”.

Al archivo ejemplo anterior hay que agregarle los campos correspondientes a los datos del tuple CorporateData en caso de indicar la acción “set”.

Par la acción “list” no se debe indicar “ID” y el archivo JSON que se retorne será un array con todos los tuples de la tabla CorporateData.

Se comunicará con el servidor SingleProxyObserverTPFI mediante un socket TCP en puerto configurable (típicamente *localhost:8080*).

## ObserverClient

Se trata de un programa escrito en Python que implementará un programa subscripto a un observador de información sobre la base *CorporateData*.

Este programa operará como cliente utilizando el siguiente formato:

```
python observerclient.py {-s=hostname} {-p=port} {-o=output.json}{-v}
```

Para hacer lo primero genera un archivo JSON indicando:

- La identificación de la CPU desde la que ejecuta (UUID).
- Acción a realizar (“subscribe”).

Se comunicará con el servidor SingleProxyObserverTPFI mediante un socket TCP al host y puerto configurable por argumento (en caso de no informar tomar *localhost* y *8080* respectivamente).

Enviaré el archivo mediante un método POST y quedará esperando la respuesta del servidor. Puede recibir múltiples respuestas por lo que deberá mantener el puerto abierto.

Cada respuesta será un archivo JSON enviado cada vez que se produzca una actualización de datos en el DB CorporateData.

Cada vez que reciba un archivo JSON lo mostraré por el archivo output.json si fue informado por argumento y por salida estándar. Luego de mostrar mantendrá abierto el puerto TCP.

En caso que el socket TCP se vea interrumpido el programa manejará la excepción re-intentando periódicamente (cada 30 segundos, parametrizable) nuevamente la conexión, de lograrla debe enviar la acción de subscripción.

El programa solo terminará por cancelación manual o terminación por excepción otra que cierre del socket pudiendo recibir múltiples notificaciones desde el observer.

### SingletonProxyObserverTPFI

Se trata de un programa escrito en Python que implementará un servidor de aplicaciones destinado a operar como un proxy para el acceso a la base de datos, un patrón singleton para el acceso a la base y un patrón *observer* para notificar a los clientes subscriptos de cambios, al mismo tiempo generará un registro de pista auditable de las acciones realizadas sobre el mismo.

El programa operará como servidor

```
python singletonproxyobserver.py {-p=port}{-v}
```

El programa aceptará conexiones TCP en un puerto a definir por argumento (en caso de no informar se tomará el \*:8080).

Cada vez que sea contactado con un cliente esperará un archivo JSON con su requerimiento.

El archivo JSON contendrá típicamente:

- El UUID del cliente que solicita.
- Una acción ("get", "set", "list", "subscribe").
- Puede contener datos adicionales según el requerimiento:
  - "get": ID del registro solicitado.
  - "set": ID del registro solicitado y datos del tuple CorporateData.

El programa soportará la siguiente funcionalidad

- Si recibe una acción "*subscribe*" (mediante POST) registra el UUID del solicitante, genera un registro de pista de auditoría en la tabla

CorporateLog indicando UUID, sesión, acción solicitada y timestamp. Luego registra el socket para uso posterior. Puede tener registrados tantos sockets como requerimiento de subscripción pudiera recibir.

- Si recibe la acción “get” (mediante POST) registra el UUID del solicitante, genera un registro de pista de auditoría en la tabla CorporateLog indicando UUID, sesión, acción solicitada, ID solicitado y timestamp. Luego forma un registro JSON con el contenido del tuple solicitado (ver Tabla a continuación) o la indicación de error, completado su envío cierra la conexión con el cliente solicitante.
- Si recibe la acción “list” (mediante POST) registra el UUID del solicitante, genera un registro de pista de auditoría en la tabla CorporateLog indicando UUID, sesión, acción solicitada y timestamp. Luego forma un registro JSON con el contenido de todos los tuples de la tabla o la indicación de error, completado su envío cierra la conexión con el cliente solicitante.
- Si recibe la acción “set” (mediante POST) espera un registro JSON con el contenido del tuple a modificar (valores indicativos en la siguiente tabla, en caso de no informar algún campo el mismo no será modificado. Si la clave no existiera previamente se creará un registro nuevo, los campos no informados se indicarán en blanco. La acción se registrará en la tabla CorporateLog indicando UUID, sesión, acción solicitada y timestamp. El registro será retransmitido al cliente solicitante y a todos los clientes que previamente se hubieran suscripto.

Tabla 1 Formato JSON indicativo para tuple CorporateData

```
{
  "id": "UADER-FCyT-IS2",
  "cp": "3260",
  "CUIT": "30-70925411-8",
  "domicilio": "25 de Mayo 385-1P",
  "idreq": "473",
  "idSeq": "1146",
  "localidad": "Concepción del Uruguay",
  "provincia": "Entre Ríos",
  "sede": "FCyT",
  "seqID": "23",
  "telefono": "03442 43-1442",
  "web": http://www.uader.edu.ar
}
```

El programa solo terminará por cancelación manual o terminación por excepción otra que cierre del socket pudiendo recibir múltiples notificaciones desde el observer.

## Requerimientos no-funcionales técnicos

La implementación de todos los programas debe ser modular y utilizando técnicas de OOP toda vez que sea posible.

En particular el programa *singletonproxyobserver.py* debe cumplir los siguientes requisitos:

- El acceso físico a la tabla *CorporateLog* y *CorporateData* debe implementarse como sendas funciones bajo sendos patrones *singleton*.
- La gestión de subscripciones tiene que implementarse como un patrón *observer*.
- La gestión de actualizaciones debe implementarse mediante un patron *proxy*.

## Arquitectura de referencia

Además de la implementación de aplicativo y clases de soporte requeridos se utilizarán los siguientes elementos de infraestructura pre-existente.

### Identificador de sesión

El identificador único de sesión informado como argumento en los distintos métodos se obtendrá mediante el método *uuid4()* del paquete *uuid* (ver ejemplos)

### Datos de CPU (UUID)

El registro de datos de CPU a utilizar en la clase *Log* pueden obtenerse del paquete denominado *platform*, con excepción del número único de CPU (UUID) que se obtiene con el método *getnode()* del paquete *uuid* (ver ejemplos).

### Acceso a base de datos.

La base de datos es de tipo *DynamoDB* y está alojada en *AWS*, el programa funciona con dos tablas *CorporateData* y *CorporateLog* las que se encontrarán previamente creadas al momento de la ejecución.  
Ver el apunte y los ejemplos provistos.

### Mensajes de trace y debug

A los efectos de verificar los valores intermedios de variables, acciones y estructuras utilizar la clase externa *logging* de tal manera que los mensajes y

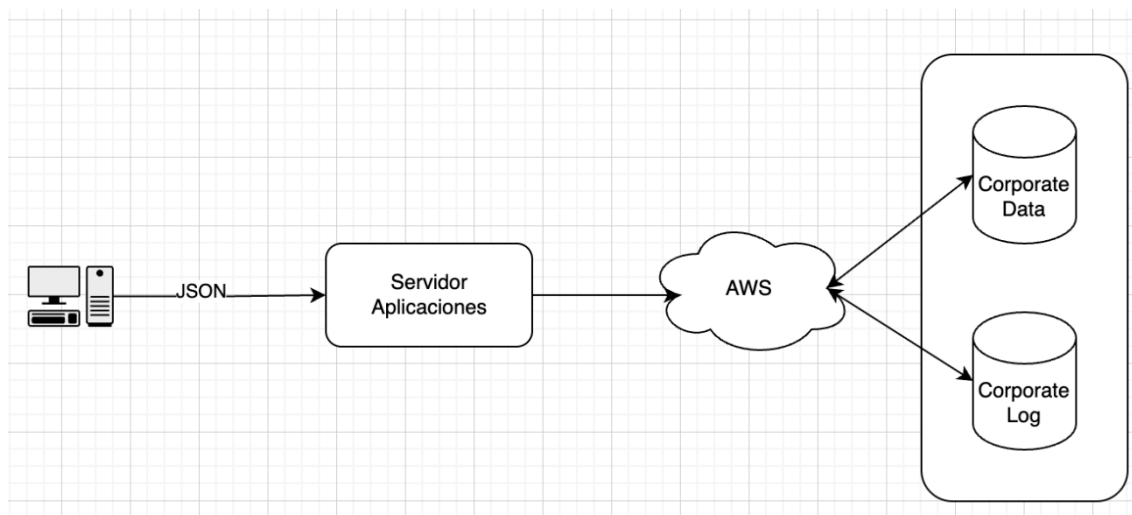
otros recursos para debug estén activados durante el test pero no al finalizar el mismo, los mensajes de ésta naturaleza deben emitirse por salida estándar. La activación se hará mediante el argumento “-v” en cada uno de los programas.

## Validación y Verificación

Se creará un test de aceptación básico automatizado con los siguientes casos de prueba:

- Camino feliz de cada una de las acciones, incluyendo registros en tabla CorporateLog de las acciones realizadas.
- Intento de llamar los programas con argumentos malformados (cada uno).
- Intento de requerimiento sin indicar datos mínimos necesarios.
- Manejo en clientes de server aplicativo caído.
- Intento de levantar dos veces el servidor de aplicaciones.

En cada test se revisará que la actividad impacte correctamente en las tablas CorporateData y CorporateLog según corresponda.



*Ilustración 2 Arquitectura de referencia*

## Consigna

La consigna perseguida por éste trabajo práctico consiste en demostrar los saberes discutidos durante el ciclo de cursado de la materia Ingeniería de Software II. Es por lo tanto importante obtener tanto un aplicativo que satisfaga los requerimientos como exhibir las técnicas utilizadas para lograrlo de manera de aplicar los conocimientos obtenidos.

En tal sentido serán necesarios los siguientes entregables:

1. Diagrama UML de actividad para gestión de list/get/set en servidor de aplicaciones (patrón *proxy*). Diagrama UML de estado para la gestión del patrón *observer* en el servidor de aplicaciones.
2. Desarrollar el código fuente de todos los componentes y librerías necesarias para implementar los requerimientos.
3. Casos de prueba consistente con lo requerido en la sección precedente Validación y Verificación en lo posible incluir logs o trazes de la ejecución de los casos de prueba indicados.
4. Los casos de prueba deberán verificar además que las clases implementadas efectivamente se implementan como los patrones requeridos. Las pruebas automatizadas deben incluir las siguientes condiciones y requisitos:
  - Todo el desarrollo lo realizará utilizando un ambiente virtual Python (venv).
  - Esqueleto de proyecto basado en CookieCutter
  - En el caso de utilizar herramientas de IA documentará los prompts utilizados en el archivo CONTEXT.md
  - La naturaleza del proyecto se mantendrá actualizada en un archivo README.md
  - Los cambios se registrarán en un archivo llamado CHANGELOG.md
  - Genere un archivo REQUIREMENTS.TXT para listar todas las librerías que pueda necesitar el proyecto para funcionar de forma de generarlas como parte de un ambiente virtual Python.
  - La licencia del proyecto será MIT y se generará documentación automáticamente con pdoc o sphynx
  - Se llevará registro de VERSION y BUILD
  - Ejecutar ruff para validar las reglas de formato y que no de errores.
  - Ejecutar black para validar el formato consistente.
  - Utilizar y validar el correcto uso de reglas de formateo PEP8.
  - Utilizar el correcto uso de convenciones PEP257 para los docstrings, aceptar solo si no hay errores.
  - Ejecutar MyPy para los modulos dentro de alcance, aceptar solo si no hay errores.
  - Ejecutar PyRight para los modulos dentro de alcance, aceptar solo si no hay errores.
  - Test unitario con Pytest con hipótesis de test unitario que permitan la cobertura del 85% o mejor.
  - Utilizar bandit para la evaluación básica de seguridad y que no queden observaciones.
  - Opcionalmente utilizar Trufflehog en los controles de seguridad.

- Producir una documentación básica del funcionamiento del módulos y actualizarla con cada PR exitoso.
- Actualizar archivo requirements.txt que se usará para las dependencias de librerías.
- Se automatizará un workflow en github integrando la fase de CI/CD completa.
- Con cada PR exitoso genera o actualiza la documentación básica con pdoc o sphinx.

Nota: Las evidencias de entregables que correspondan a texto pueden insertarse en el cuerpo del reporte con un font no proporcional (Courier New por ejemplo) mientras que la evidencia de las capturas de pantalla puede adjuntarse la imagen respectiva referenciando claramente a que condición corresponde.

El trabajo práctico final integrador deberá realizarse en forma individual. El sistema sobre el que se lo implementará registrará el identificador único de cada CPU donde se ejerce el trabajo de manera de auditar tal condición. La cuenta AWS que contiene las tablas involucradas tiene activado el módulo AWS de auditoría para registrar IP y MAC addr de los accesos realizados.